5. Design

5.1 Model Design

Collection: The *Collection* object can be created and deleted. When created, it is given a name. It contains an array of game IDs.

Review: The *Review* object can be created and deleted. When created, it is associated with a *Game*. It contains a star rating and may contain a review that the user writes. All reviews are stored within the *User Database*.

Game: The *Game* object contains a description of the game, the image of the game, the game ID, its categories, and its mechanics.

User: The *User* object can be created and deleted. Once created, the object can add and delete personal *Collections*. Users also given the privilege of *isAdmin()* have the ability to delete users from the *User Database*.

User Database: The *User Database* object contains all user data, review data, and privileges. It is also responsible for writing and updating the User XML file.

Game Database: The *Game Database* object contains all games and their data. *Games* within the Game *Database* can be manipulated to fit criteria through a filter.

FileScanner: The *File Scanner* object will be responsible for reading the XML data for *User Database* and *Game Database.*

5.2 View Design

Side Panel: The Side Panel will be applied to all screens through the application. It provides buttons for navigating to the *Library Screen*, *Setting Screen*, *Logout*, and *Home Screen*.

Game Panel: The Game Panel will populate a list of games depending on the results of *filterByMechanics()* and *filterByCategory()*. This panel appears on the left side of the screen alongside the *Add Game Panel* when a filter or a category search is executed.

Add Game Panel: The Add Game Panel will populate on the right side of the *Game*

Panel when *filterByCategory()* or *filterByMechanics()* is called. This panel displays all the user's collections by calling *getCollections()*. The user can drag a game from the *Game Panel* and drop it into the desired collection in the *Add Game Panel,* calling *addGame(gameId)* onto the chosen collection.

Login Screen: The Login Screen will present a *usernameField* and *passwordField* for the user to enter credentials. *User Database* will call *validateCredentials()* before letting the user enter the application. If an individual wants to create an account, the *createAccount* button will take them to the *Create Account* screen.

Home Screen: The Home Screen will display different games at different times of the day through the *gamesOfTheDay* panel. The *searchField* will let the user query games, and *searchByName()* will be called before taking the user to the *Search Screen* with results. The *BrowseBy* button will display available categories. When a category is selected, the user will be redirected to the *Search Screen*. Upon attempting to log out of the session, the *Save Changes Panel* will pop up, asking the user if they would like to save their changes. If confirmed, *UserDatabase.writeToXML()* will be called to update any changes made. The *Side Panel* will be present to allow the user to navigate to other pages.

Account Creation Screen: The Account Creation Screen will display a *usernameField*, *passwordField*, and a *verificationField* for the user to confirm their password. After verifying that the passwords match, the *User Database* will call *isUsernameTaken()* to make sure the desired name is unique. After confirmation, the *User Database* will call *addUser()* to create a new user account and also write to the XML to update the changes. The user is then redirected to the *Home Screen.*

Library Screen: The Library Screen displays all the collections belonging to a specific user. The *collectionPanel* will generate all of a user's collection when *getCollections*() is called. The *deleteGameButton* will call on *deleteCollection*() from the *User* class, removing a collection from the Library Screen. Afterwards, if the user decides to save their changes, *UserDatabase.writeToXML()* is called to update the information. The *Side Panel* will be present to allow the user to navigate to other pages.

Collection Screen: The Collection Screen displays all games belonging to a specific collection. It will contain a *gamesPanel* that generates when *getGameIds()* is called and through the *Game Database* class *getGamebyId()*. The *deleteGameButton* will call on the *removeGame()* method from *Collections* to remove a specific game. Afterwards, if the user decides to save their changes before exiting the session, *UserDatabase.writeToXML()* is called to update the information. The *Side Panel* will be present to allow the user to navigate to other pages.

Search Screen: The Search Screen will display all games from the given search criteria. There is the *searchField* if the user decides to query through text. It will trigger *the searchByName() function in the Game Database*. The *BrowseBy* button will allow users to call *filterByCategory or filterByMechanics.* The *gamePanel* will generate games resulting from the decided search option. The *Side Panel* will be present to allow the user to navigate to other pages. If the user selects *filterByCategory()* or *filterByMechanics(),* once entering their search query, the *gamePanel* will be generated alongside the *addGamePanel* so the user can drag and drop their desired game into a collection.

Game Description Screen: The Game Description Screen will display the game, all its information, and reviews related to it. It will retrieve the game's information through the *Game* class's getter functions and populate the *descriptionPanel* with all information related to the game. The *imageTag* will display the game's photo by calling *getImageTag*() from *Game*. When the user is ready to add a game, the *AddGame* button will let the user select their desired collection. The *reviewDisplay* panel displays existing reviews pulled from the *User Database*. On the panel, there is an *AddReview* button that the user can use to submit a new review. A new Review object will be created in the *reviewPanel*. The method *addReview()* will add the review to the user object, and also *the UserDatabase.writeToXML()* will be called to update the changes. The *Side Panel* will be present to allow the user to navigate to other pages.

Manage Users Screen (Admin): The Manage Users Screen is accessed by the Admin account, which is verified through *User.isAdmin()*. The screen generates buttons for all the users by calling *User.getUsername()*. The *manageUser* panel will list all accounts. When the admin selects a user and clicks delete, which is triggered through *Admin.deleteUser(),* the *User Database* will perform *writeToXML()* to update the file. The *Side Panel* will be present to allow the user to navigate to other pages.

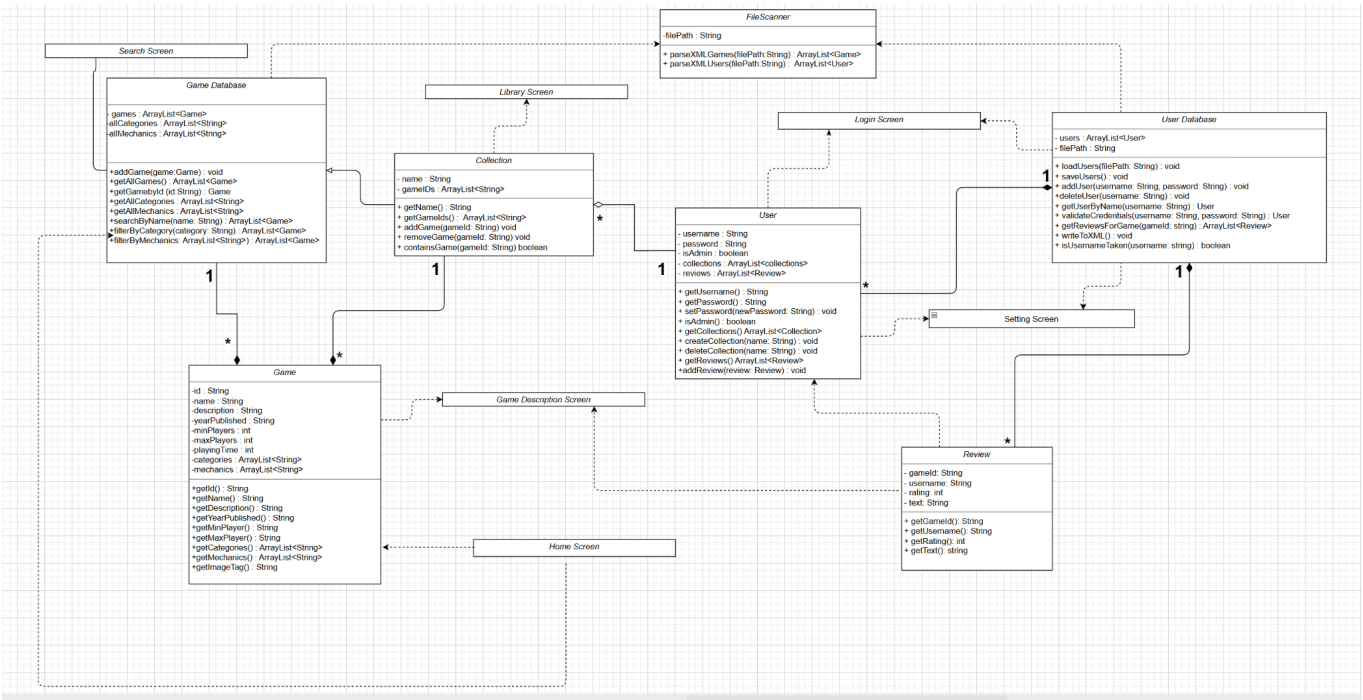
Setting Screen: The Setting Screen will display the user's name.

5.3 Control Design

a. Logging into the system. When a user logs into the system, the account they choose will determine what collections are loaded into the system.
b. Adding a game to a collection. Add a game to a collection list to update and repopulate the collection's view.
c. Removing a game from a collection. Remove a game from a collection list to update and repopulate the collection's view.
d. Adding a collection. Adds a collection to the library page to repopulate the user's library view.

e. Deleting a collection. Remove a collection from the *Library Screen* to repopulate the user's library view.
f. Filtering a search. There are three ways to filter a search (search by name, by filter, or by category). Once a search query has been executed, the search page generated will be based on the criteria provided.
g. Save changes. When the user selects the save button, *User Database* will keep up with persisting changes updating the XML file holding all users' information.
h. Adding a Review and Rating. Adding a review and rating will update the *User Database.*
i. Creating an account. When a user creates a new account, *User Database* will add a new user object and write to the XML file.
j. Deleting an account. When an admin deletes a user account, *User Database* will remove the *User* object and the associated information with it before rewriting to the XML file to reflect the changes.
k. Logging out of account. When a user logs out, any unsaved changes are prompted to be saved before returning to the *Login Screen*. Any unsaved changes will be cleared from memory.

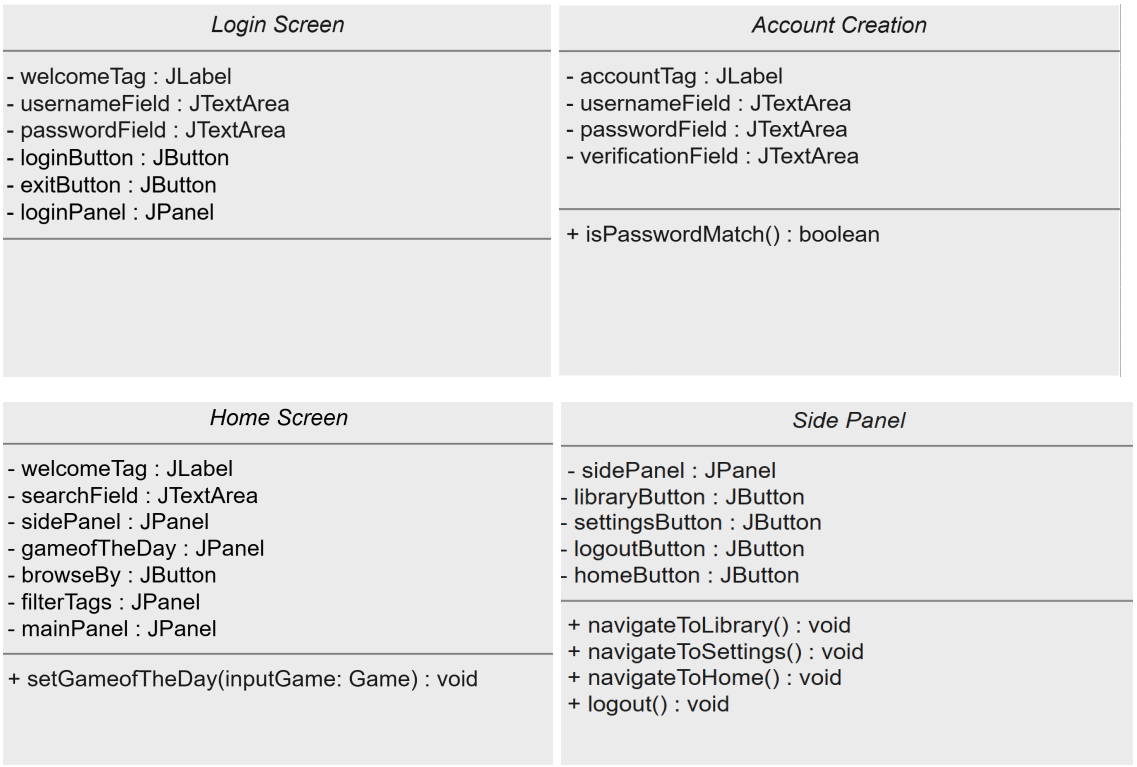
5.4 Communication Design



The class diagram above consists of the main classes for AB Board Games Geek. Users log in through the GUI, which communicates with the *User Database* to confirm the user. FileScanner is responsible for reading the XML data to populate the user information and

the Game Database. Once logged in, users will arrive at the *Home Screen* where they can search for games by name through the Game Database's *searchByName()* function. They can also search for games through mechanics and filters with *filterByCategory()* and *filterByMechanics()*. Once entering a query, the user will be shown all relevant games on the left-hand side through the *Game Panel*, and the *Add Game Panel* appears on the right, allowing users to drag and drop games into their desired collection. Users can manage their *Collections* through the *Library Screen* , which will allow them to create and delete *Collections*. *Collection* objects will store game IDs that reference games in the *Game Database*. On the *Game Description Screen*, users can view specifications about the selected game and leave a *Review,* including a star rating and optional text. *Review* objects are stored in the *User Database*. Users who have administrative powers, which are granted through *isAdmin()* are given the ability to delete user accounts from the *User Database.* All changes made by the user will be asked to be saved before exiting the Board Game application. They are persisted by the *User Database* through *writeToXML()* to keep the user XML updated.

GUI Classes



Library Screen

- sidePanel : JPanel
- collectionPanel : JPanel
- libraryPanel : JPanel
- deleteCollectionButton : JButton

Collection Screen

- sidePanel: JPanel
- gamePanel: JPanel
- deleteGameButton: JButton

Search Screen

- sidePanel : JPanel
- searchField : JTextArea
- gamesPanel : JPanel
- addGamePanel : JPanel
- BrowseBy : JButton
- filterTags : JPanel

AddGamePanel

- addGamePanel: JPanel
- collectionDisplay: JPanel

+ displayCollections(collections: ArrayList<Collections> : void
+ addGameToCollection(gameId: String, collection:Collection) : void

Game Description Screen

- sidePanel: JPanel
- description: Jpanel
- imagetag: Jlabel
- gameName: Jlabel
- reviewDisplay: JPanel
- addReview: JButton
- addGame: JButton

+ setDisplayedGame (inputGame : Game): void

Admin Setting Screen

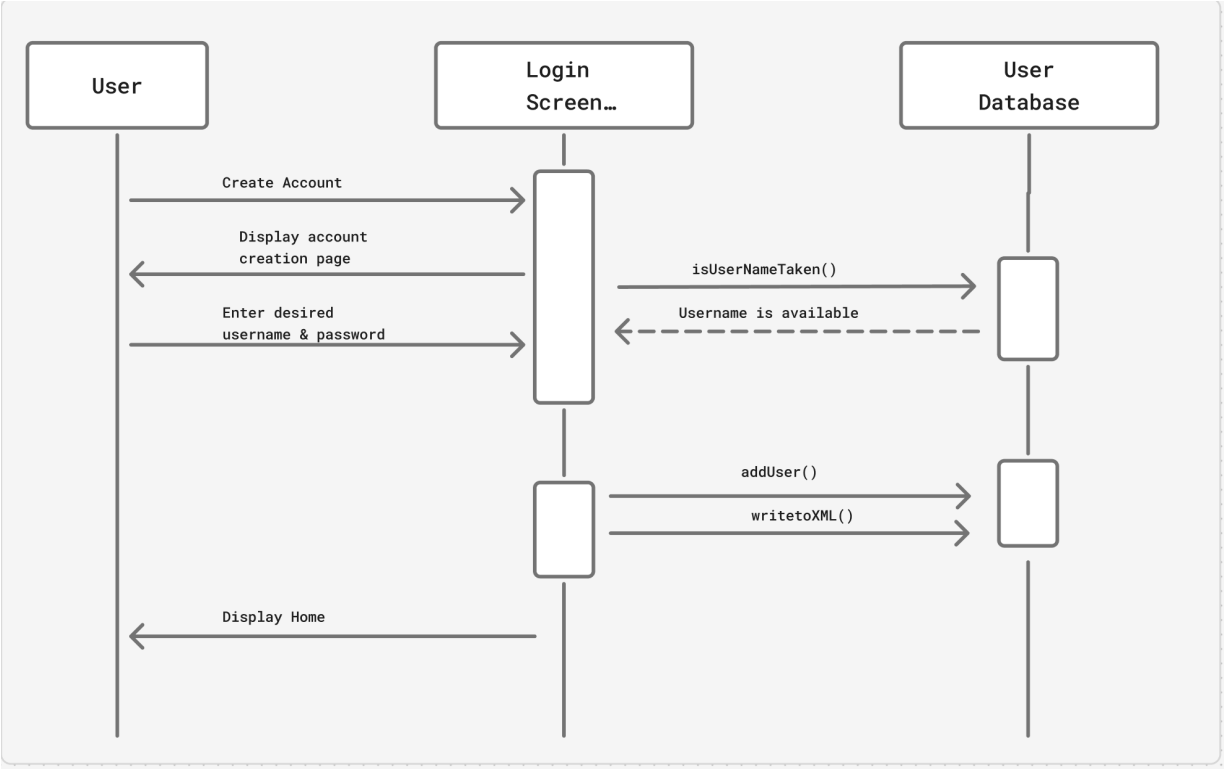
-sidePanel : JPanel
-usernameTag : JLabel
-manageUser : JButton
-viewAccountsPanel : JPanel

Settings Screen

-settingPanel : JPanel
-usernameTag : JLabel

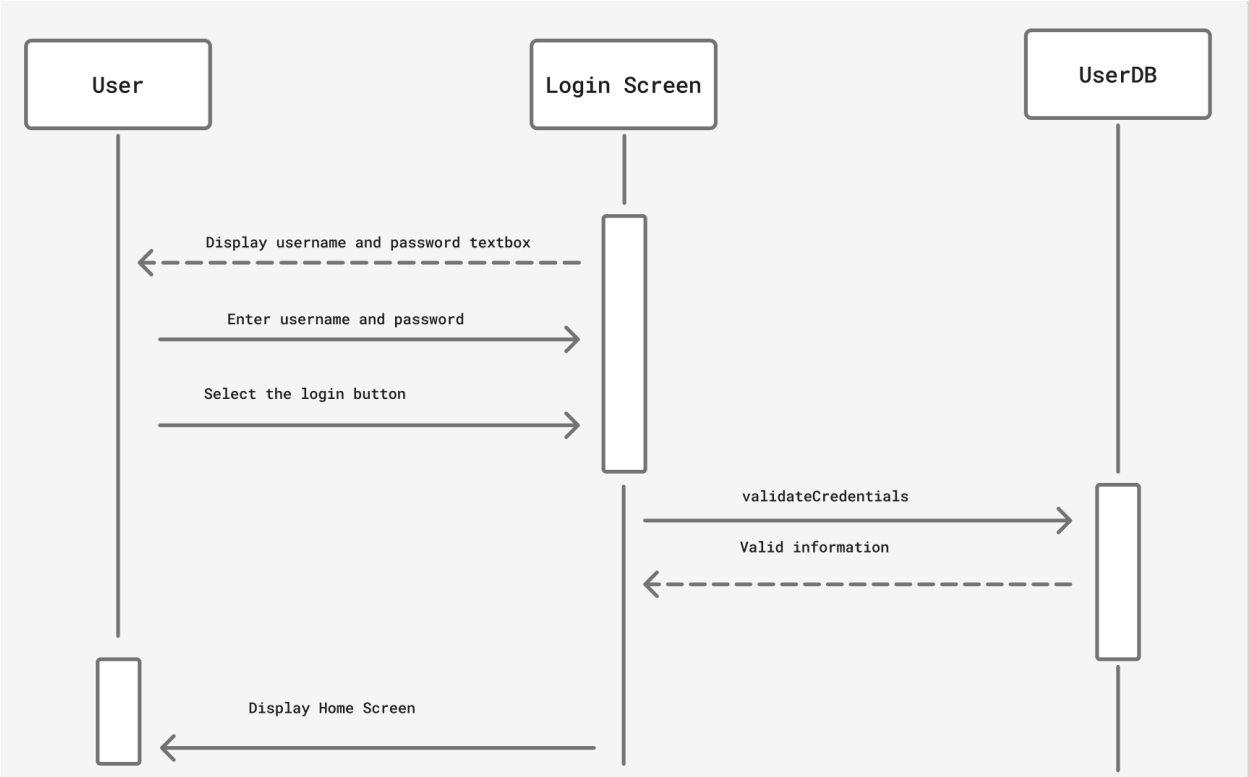
Activity Diagrams

Account Creation



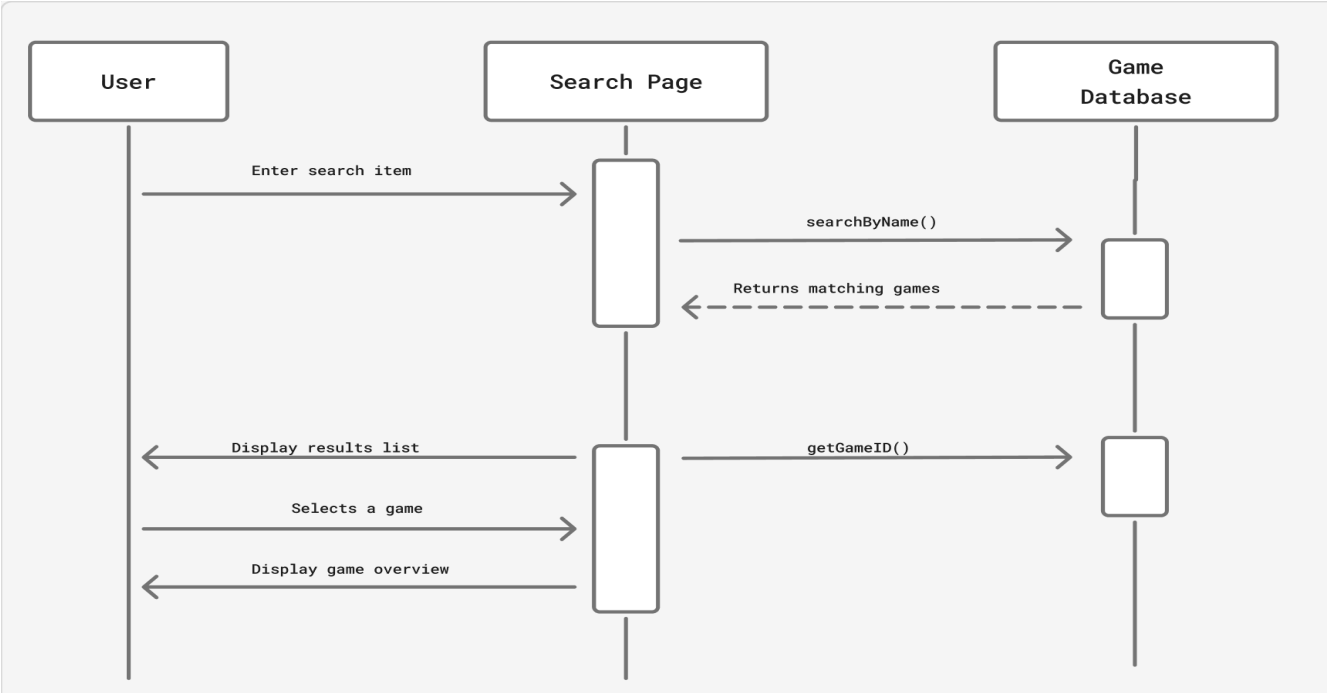
A user clicks *Create Account* on the *Login Screen,* and the GUI navigates to the *Account Creation* page. The user can then input their username and password into the textboxes. They can then select *Confirm,* which will trigger the function *isUsernameTaken*. If the username is taken, the function will return -1, and the GUI will tell the user that the username is taken. If the username is not taken, the function will return 1. The function *addUser()* will be called, and the GUI will tell the user that the account creation is successful, displaying the *Home Screen* afterwards.

Login



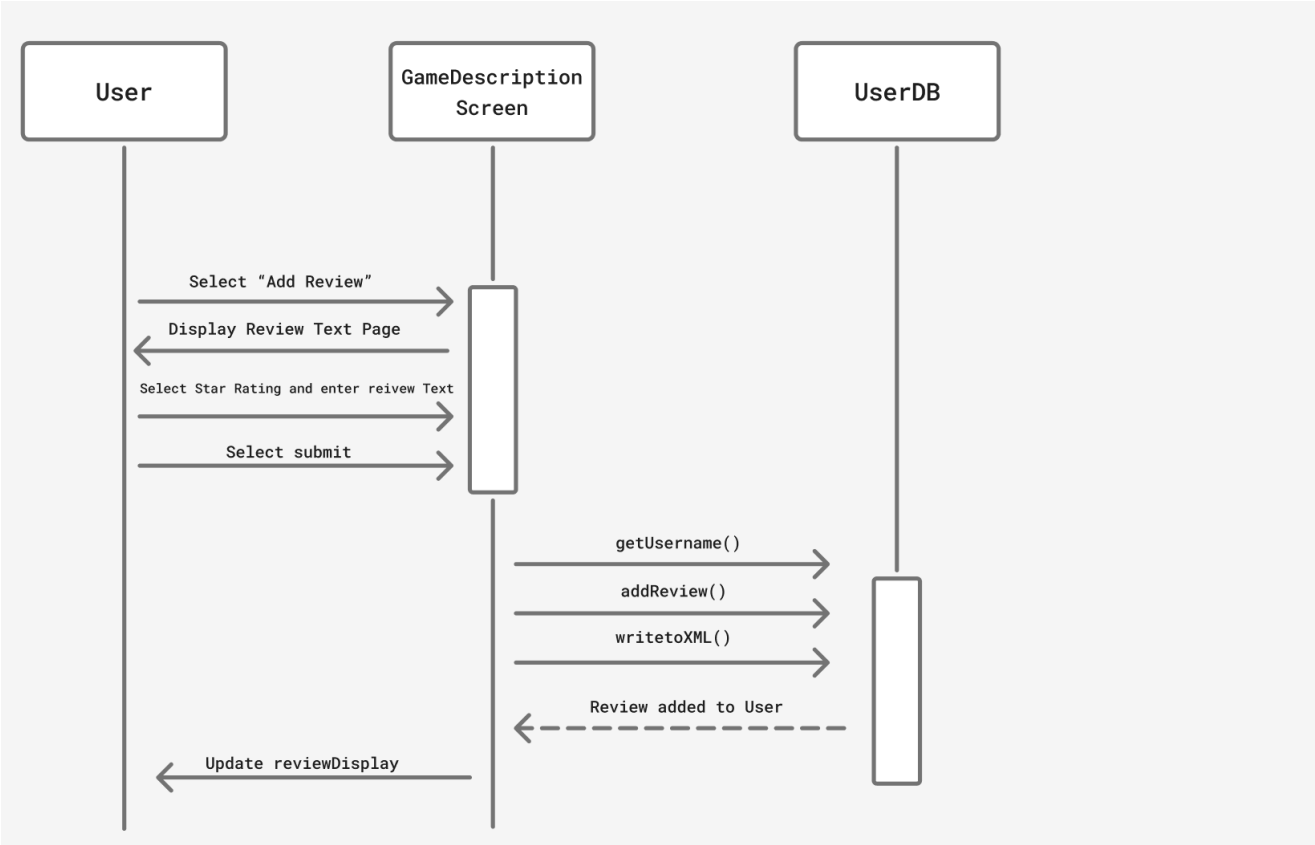
A user logs in with their credentials. After selecting L*ogin*, it triggers the function *validateCredentials()* from the User *Database* to check if the user has entered their information correctly. If the username is not found or the password is not associated with the username, the function will return -1 and show an error. Otherwise, the function will return 1. If the login is successful, the GUI will display the *Home Screen*.

Searching for Game



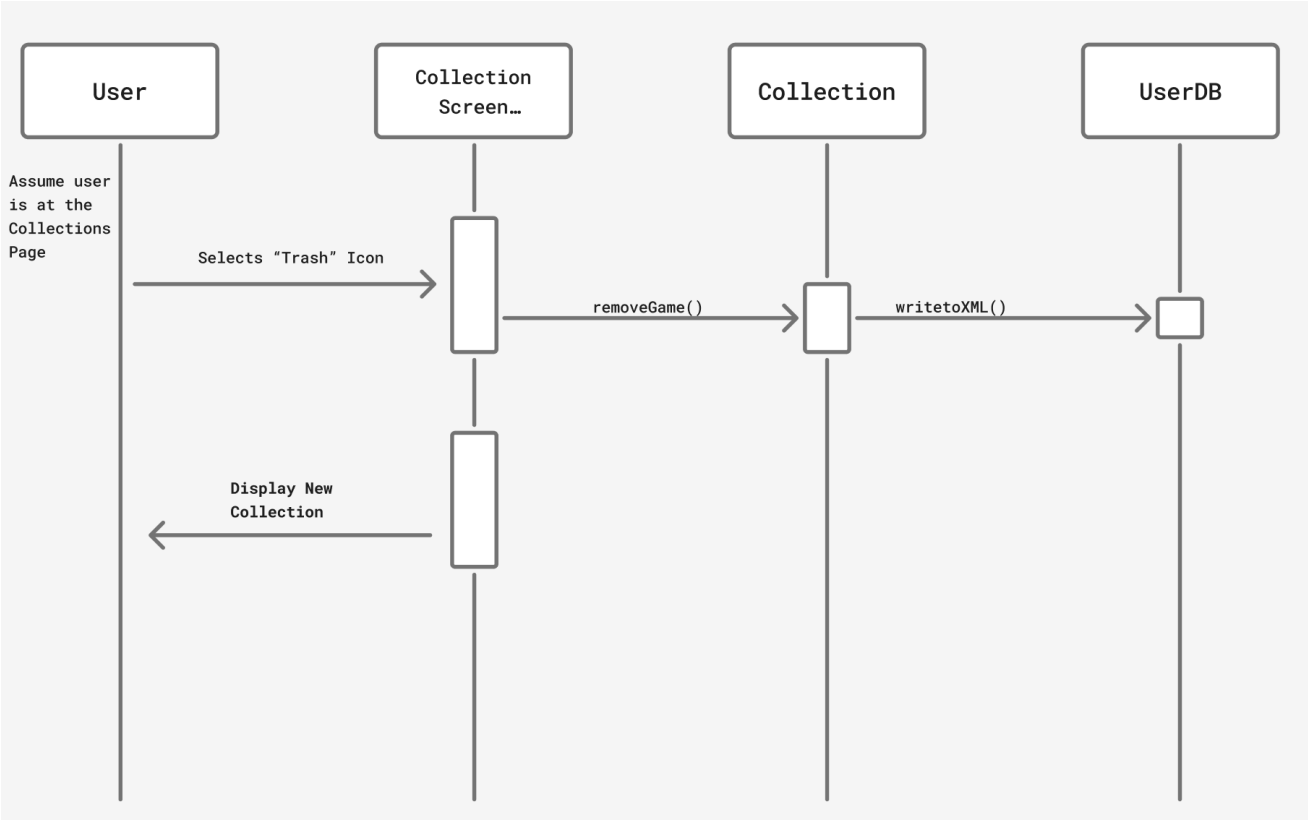
There are multiple ways for a user to search for a game. On the *Home Screen*, the user can enter their query in the *searchField.* When the user clicks enter, this selection triggers the function *searchByName()* from *Game Database* to generate the games related to the search query. There are two other features under the *BrowseBy* button: *filterByMechanics* and *filterByCategory*. This will create a dropdown menu, and the user can choose different options to filter their games by.

Add Review and Rating



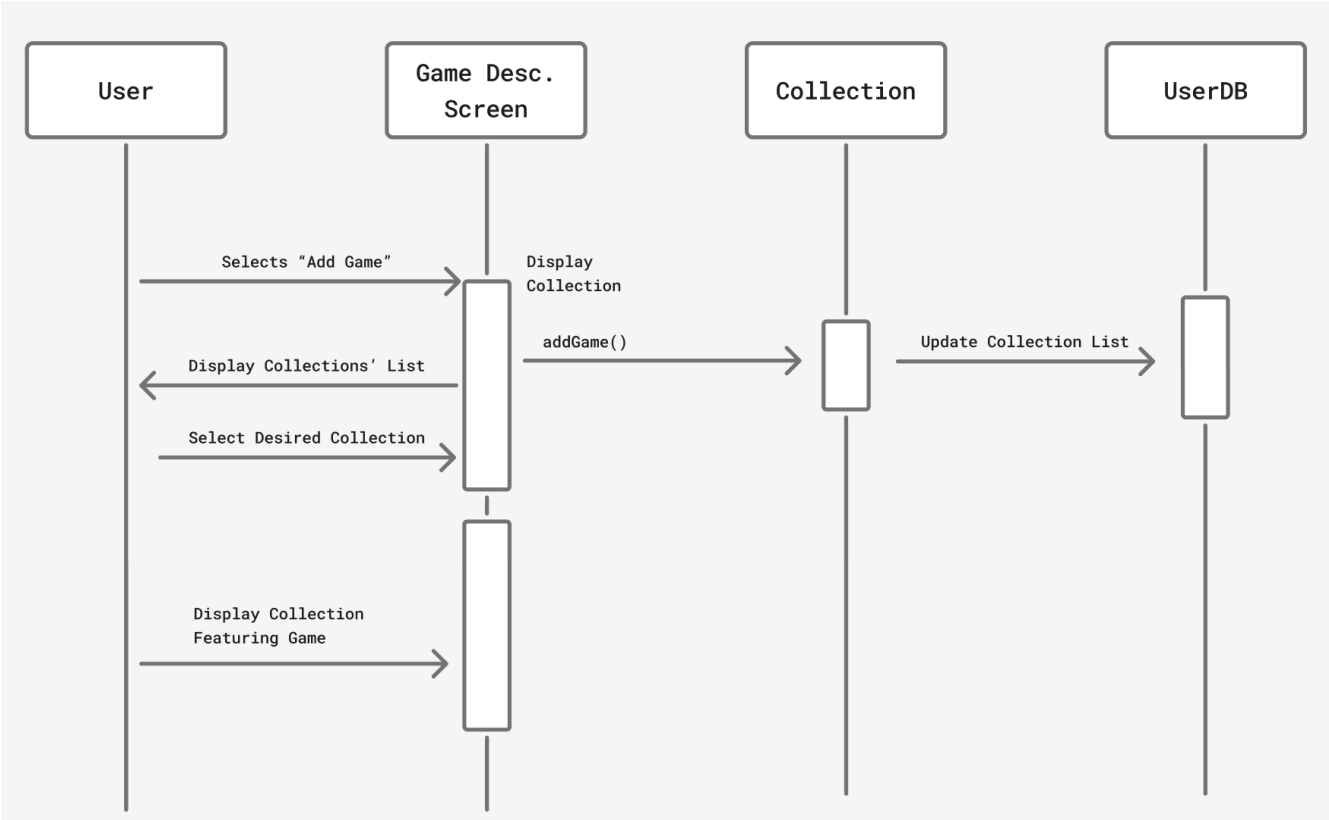
When the user is looking at a game description, they can select the *Add Review* button. The GUI will then take the user to the *reviewDisplay,* which has a textbox and an option for entering a number of stars, the rating. Once the user has filled out the number of stars and written a review (optional), they can then select the *Submit* button. The *getUsername, addReview,* and *writetoXML* functions will be called, and then the review will be added to the user database.

Deleting A Game From A Collection



This activity diagram assumes that the user is already on the *Collection Screen*. To remove a game from a collection, the user selects the *trash* icon. The GUI then displays the collection without the removed game. The *trash* icon calls *removeGame()* on the collection, which will remove the game from the collection's list.

Adding A Game To A Collection



This activity diagram assumes that the user is on the *Game Description Screen*. To add a game to a collection, the user will select the *Add Game* button. The GUI will then display the user's collection list, allowing the user to choose which collection they would like to add the game to. Once a collection is selected, the system will call addGame() on that collection.